Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

```
Environment Setup
To set up your environment, follow these steps:

 Create a new project directory:



bash
mkdir mongoose-schemas-lab
cd mongoose-schemas-lab

 Initialize a new Node.js project:



bash
npm init -y

    This command creates a package.json file with default settings.
 Install required packages:



bash
npm install express mongoose body-parser

    This installs express for creating the web application, mongoose for
interacting with MongoDB, and body-parser for parsing request bodies.
 Create index.js file:


    Create a file named index.js in your project directory. This file will
contain the main application code.
 Project Structure:


    Your project structure should look like this:


mongoose-schemas-lab/
├── node_modules/
├── index.js
└── package.json

Step-by-Step Instructions
Step 1: Connect to MongoDB
```

First, establish a connection to your MongoDB database using Mongoose.

Open index.js and add the following code:

```
javascript
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const app = express();
const port = 3000;

// Middleware to parse JSON bodies
app.use(bodyParser.json());

// MongoDB connection string
const dbURI = 'mongodb://127.0.0.1:27017/bookstore';

mongoose.connect(dbURI, {
useNewUrlParser: true,
useUnifiedTopology: true
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('MongoDB connection error:', err));

app.listen(port, () => {
console.log(Server is running on port ${port});
});
```

Explanation:

This code imports the necessary modules (express, mongoose, and body-parser).
It defines the connection string dbURI for your MongoDB database. Make sure your MongoDB server is running. If your database name is different from bookstore, change it accordingly.
mongoose.connect() establishes the connection to MongoDB.
The .then() and .catch() blocks handle successful connection and connection errors, respectively.
* It starts the Express server on port 3000.

Run the application:

```
bash
node index.js
```

You should see "Connected to MongoDB" in the console if the connection is successful.
Step 2: Define the Book Schema
Next, define the Mongoose schema for the Book model.

Add the following code to index.js:

javascript

```
const bookSchema = new mongoose.Schema({
title: {
type: String,
required: true,
trim: true,
maxlength: 100
},
author: {
type: String,
required: true,
trim: true
},
genre: {
type: String,
enum: ['Fiction', 'Non-Fiction', 'Science Fiction', 'Mystery'],
default: 'Fiction'
},
publicationYear: {
type: Number,
min: 1800,
max: new Date().getFullYear()
},
price: {
type: Number,
required: true,
min: 0
},
createdAt: {
type: Date,
default: Date.now
}
});
const Book = mongoose.model('Book', bookSchema);
```

Explanation:

bookSchema defines the structure for the Book model.
title: A string that is required, trimmed, and has a maximum length of 100 characters.
author: A string that is required and trimmed.
genre: A string that must be one of the specified enum values, defaulting to 'Fiction'.
publicationYear: A number that must be between 1800 and the current year.
price: A number that is required and must be greater than or equal to 0.
createdAt: A date that defaults to the current date and time.
mongoose.model('Book', bookSchema) creates a Mongoose model named Book using the defined schema.

Step 3: Create a Route to Add a New Book

Implement an Express route to handle the creation of new books.

Add the following code to index.js:

```
javascript
app.post('/books', async (req, res) => {
try {
const book = new Book(req.body);
await book.save();
res.status(201).send(book);
} catch (error) {
res.status(400).send(error);
}
});
```

Explanation:

This route handles POST requests to /books.
It creates a new Book instance using the data from the request body (req.body).
book.save() saves the new book to the database.
If successful, it sends a 201 status code (Created) and the saved book as a response.
* If there's an error (e.g., validation failure), it sends a 400 status code (Bad Request) and the error message as a response.

Step 4: Test the Route

Use a tool like Postman or curl to test the /books route.

Open Postman or your preferred API testing tool.

Create a new POST request to http://localhost:3000/books.

Set the Content-Type header to application/json.

Add the following JSON to the request body:

```
json
{
"title": "The Hitchhiker's Guide to the Galaxy",
"author": "Douglas Adams",
"genre": "Science Fiction",
"publicationYear": 1979,
"price": 10.99
}
```

Send the request.

Expected Output:

You should receive a response with a status code of 201 and the saved book data, including the _id field generated by MongoDB:

```
json
{
"title": "The Hitchhiker's Guide to the Galaxy",
"author": "Douglas Adams",
"genre": "Science Fiction",
"publicationYear": 1979,
"price": 10.99,
"createdAt": "2024-10-27T14:00:00.000Z",
"_id": "653c0a3e6e1b7d001b8f2a5c",
"__v": 0
}
```

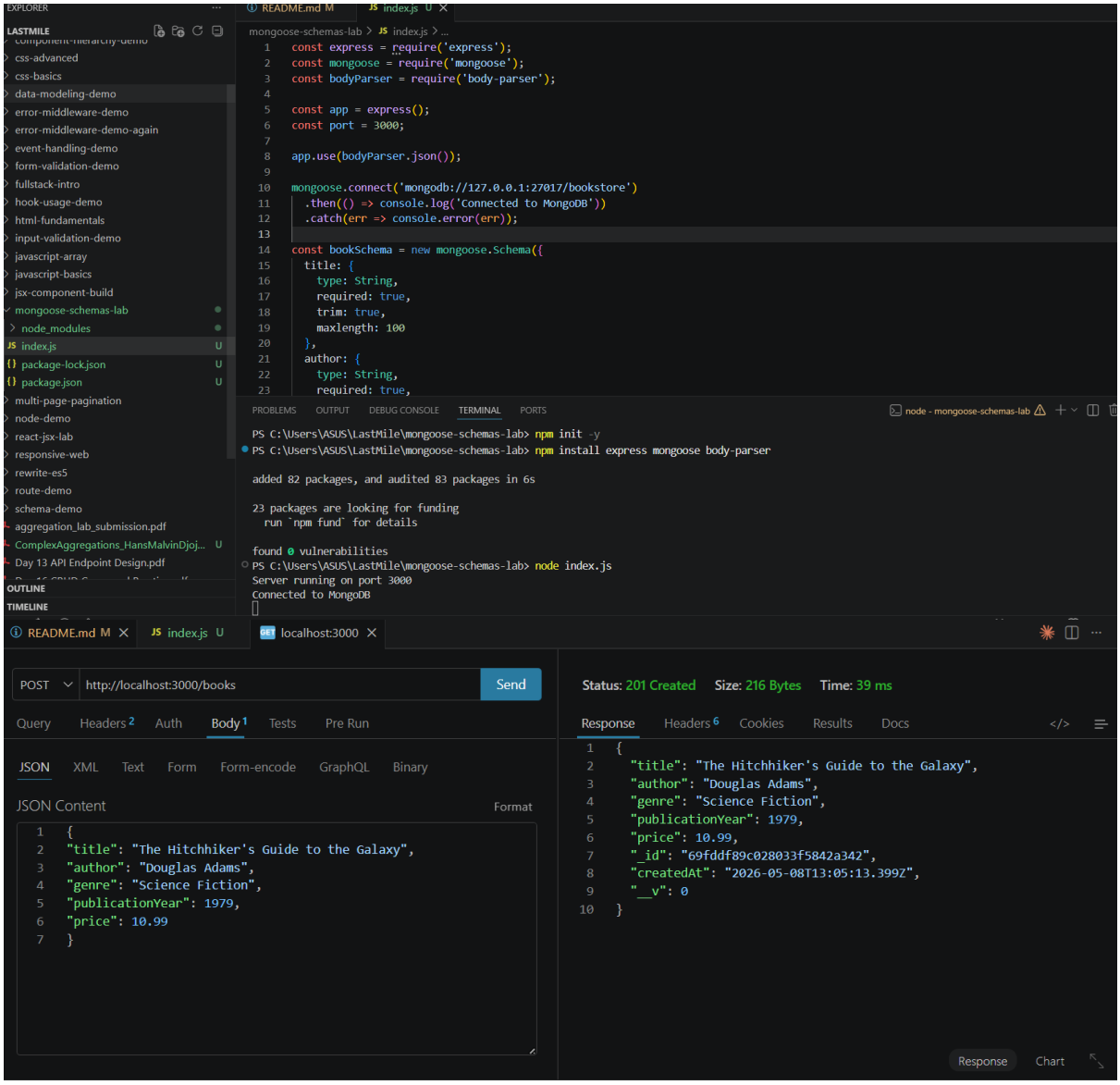
Testing Validation:

Try sending a request with invalid data, such as a missing title or an invalid genre. For example:

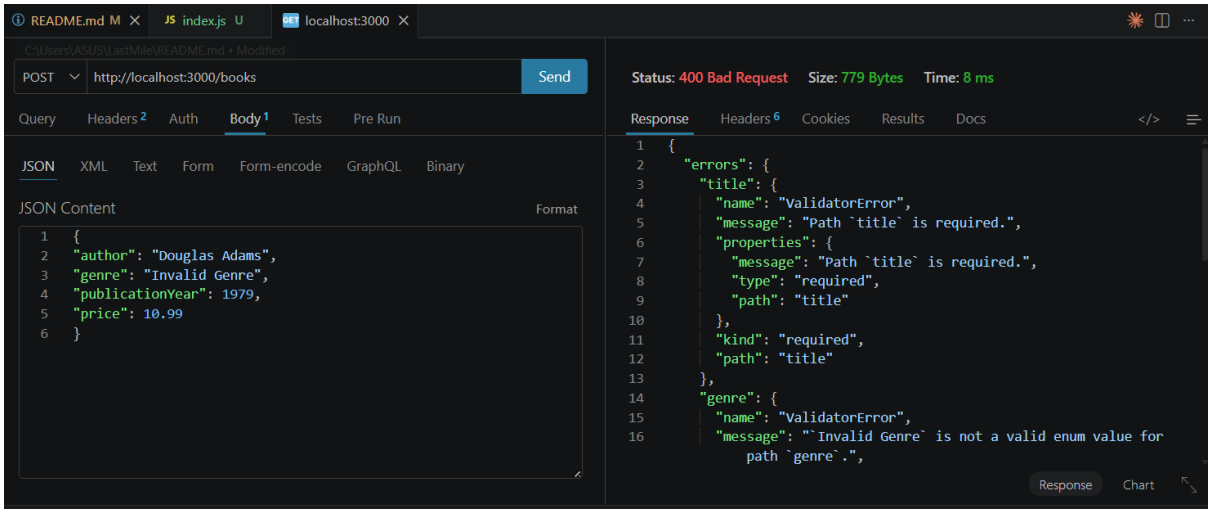
```
json
{
```

```
"author": "Douglas Adams",
"genre": "Invalid Genre",
"publicationYear": 1979,
"price": 10.99
}
```

Answer:



Validation test for error



I verified the data persistence by accessing the bookstore database through the mongosh terminal and executing the db.books.find() command to retrieve all stored records. This allowed me to confirm that each book document appeared in the collection with the correct schema fields and the unique _id automatically generated by MongoDB. Alternatively, I used a GUI like MongoDB Compass to visually inspect the books collection, ensuring that the documents were properly saved and reflected the data sent via the POST request.